

Credit Risk and Portfolio Exposure Analysis on a Synthetic Banking Dataset

A Portfolio Project in Credit Risk and Data Quality Assessment

Tine Lisec, Recent graduate from

University of Ljubljana - School of Economics and Business

field of study: Money & Finance

September 2026

Contact

tine.lisec@gmail.com

+386 (0) 51 760 222

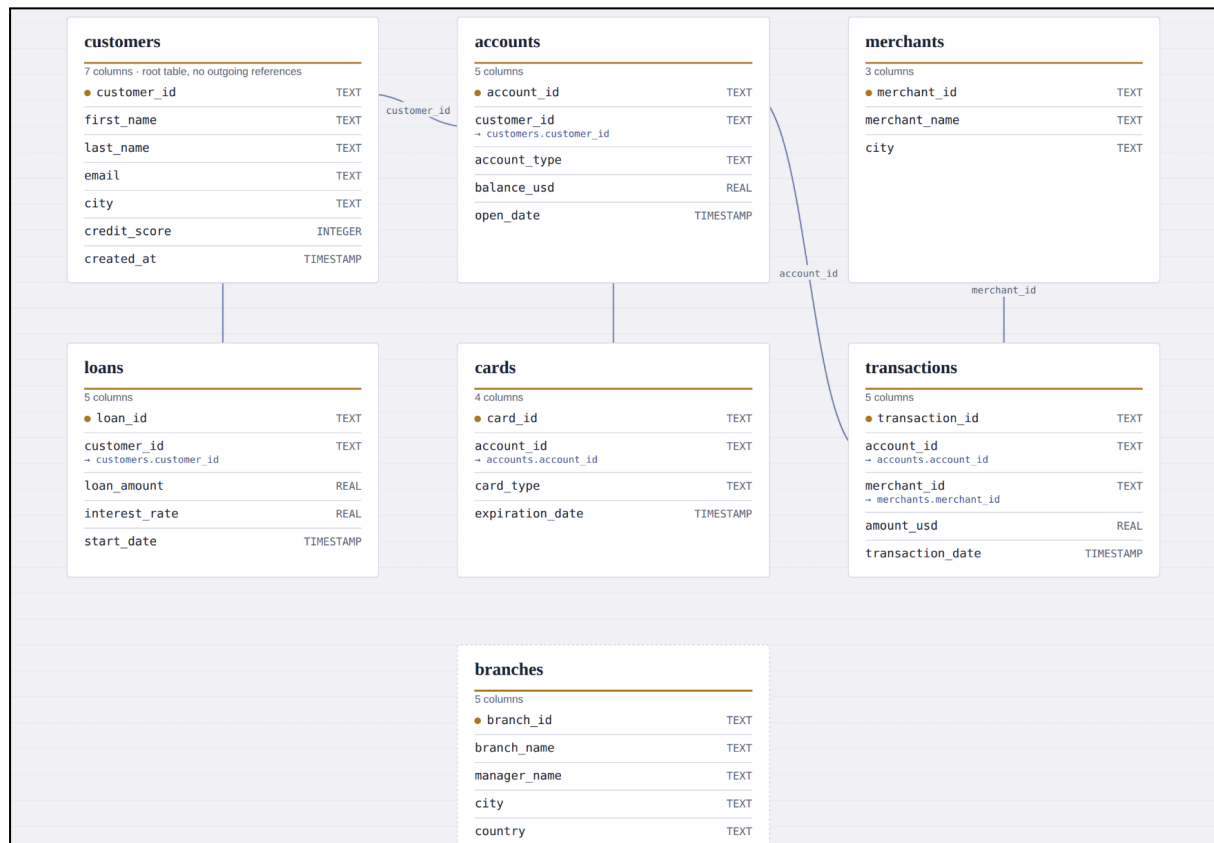
Abstract

This report is a self-directed bank portfolio project demonstrating practical risk analysis skills for finance internship applications, using a synthetic commercial banking dataset of 1.26 million records across customers, accounts, loans, cards, transactions, and merchants. The dataset shows clear signs of synthetic generation and carries real limitations, including missing fields like loan status and default history, flagged upfront in Section 2 rather than left to surface partway through. The goal is to show the analytical process end to end: checking data quality before trusting it, applying risk vocabulary correctly, and being explicit about where the analysis's boundaries sit. All data collection, calculations, and visualizations were built using SQL and Python.

Section 1: Data Overview & Quality Notes

This dataset contains synthetic data from a commercial bank, covering six connected tables (customers, accounts, loans, cards, transactions, merchants) plus a separate, unconnected branches table. Customers open accounts and take out loans, accounts have linked cards, and spending at merchants is recorded as transactions.

Figure 1: Database Schematic



The data spans January 1, 2019 (the first customer profile created) through December 31, 2025 (the last recorded transaction), with 50,000 customers, 75,000 accounts, 30,000 loans, 100,000 cards, 1,000,000 transactions, and 500 branches. Since the dataset gives no indication of whether a loan is still active, and no loan duration field exists, **all loans are assumed active for this analysis**.

Before moving to analysis, a few data quality issues are worth flagging.

First, city and country are null in every single row of the dataset, so no geographic analysis at the customer level is possible.

Second, duplicate records are (possibly) common: 6,300 of 50,000 customers (12.6%) share a full name with at least one other customer (which could also be a coincidence), and 3,390 share an email address, which **likely reflects how the data was generated rather than genuine duplicate customers**. **Third**, and most

importantly, the dataset shows clear signs of being synthetically generated rather than reflecting real bank activity: **transaction timing is spread almost perfectly evenly across days of the month and hours of the day**, with no spikes around typical payroll or rent dates or business hours; **transaction amounts follow a uniform distribution across leading digits rather than the pattern predicted by Benford's Law**, which real financial data reliably follows; and **average transaction value, volume, and concentration** (Gini coefficient of 0.333) are **nearly identical across savings, checking, and business accounts**, which would be highly unusual in a real customer base.

Figure 2: Volume density Heatmap

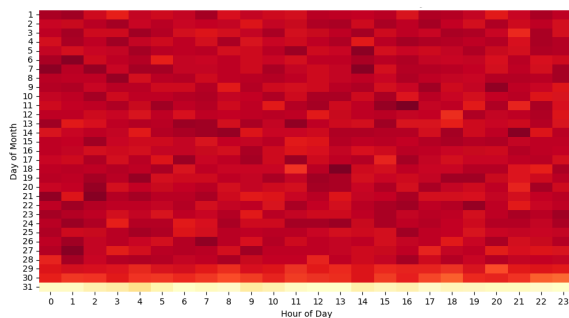
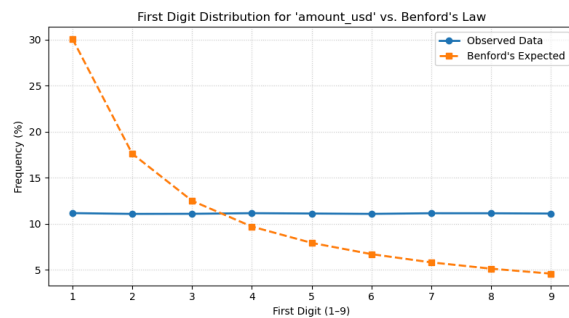


Figure 3: Benford's Law test



These patterns don't undermine the dataset's usefulness for practicing analytical methods, but they mean findings should be read as illustrative of technique rather than reflective of real-world banking behavior.

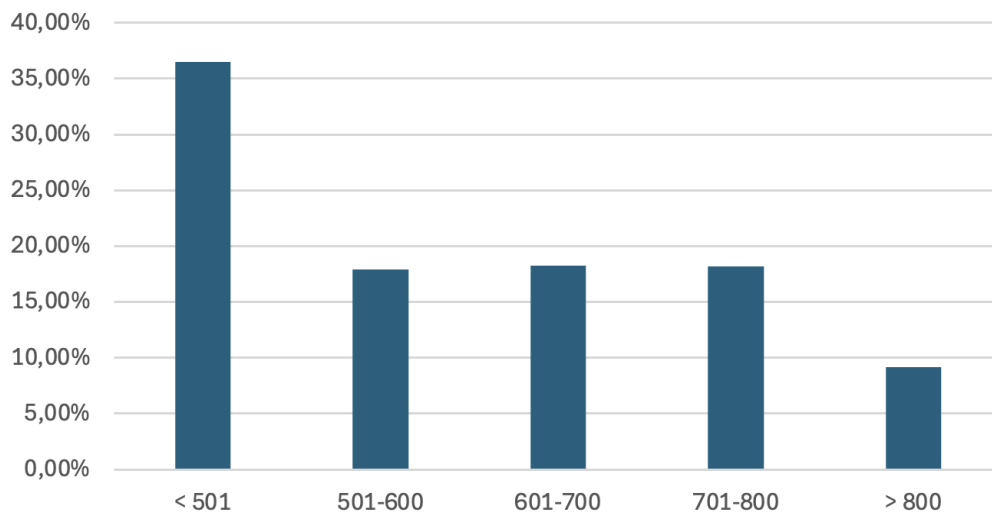
Section 2: Customer & Account Base

Before assessing credit risk, it helps to establish who the customer base actually is. Credit scores split into five bands (visible on the chart below): 36.5% of customers score below 501, 17.9% fall in 501-600, 18.3% in 601-700, 18.2% in 701-800, and 9.1% above 800. At first glance that looks uneven, but the bands aren't the same width, the below-501 band spans roughly 200 points while the above-800 band spans only about 50. Once you account for that, the density per point is the same across the whole score range, confirming the same flat, uniform pattern already flagged in Section 2, rather than the bell-curve concentration around a mid-range score you'd expect from a real customer population.

Table 1: Top 10 cities with the highest number of customers

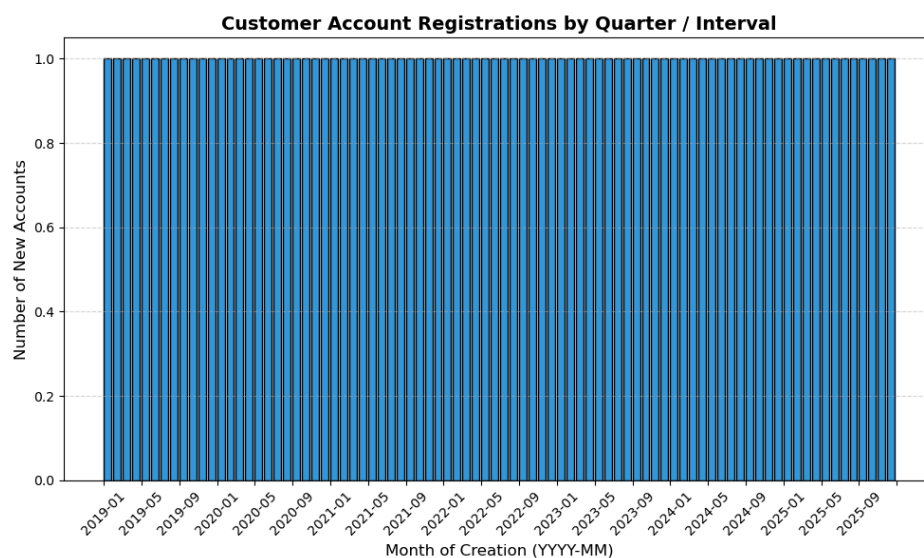
City	Number of customers
South Michael	56
East Michael	42
New Michael	41
Port Michael	39
New Jennifer	39
Smithmouth	38
New David	37
Lake Michael	37
New John	35
West Michael	34
Total customers in top 10 cities	398

Figure 4: Share of customers in each credit score bucket



On geography, customers are spread across 25,112 unique cities, and the ten most common cities combined account for only 0.796% of the total customer base, so location isn't a meaningful segmentation variable here. Account types split almost exactly into thirds: business accounts make up 33.1% of the total, with checking (33.4%) and savings (33.5%) splitting the remaining personal accounts nearly evenly. Account tenure tells the same story. Accounts were opened at a steady, unbroken pace across the full seven-year window from January 2019 to December 2025, with no visible peaks around economic events, seasonal cycles, or product launches, and no dips during slow periods. That steadiness is itself a signal: a real bank's account-opening volume tracks marketing campaigns, market conditions, and branch expansion, none of which appear here.

Figure 5: Account tenure, too uniform to be realistic.



Section 3: Portfolio Composition & Concentration

Portfolio exposure is spread about as evenly as everything else in this dataset. Total balances split fairly evenly across account types: checking holds \$2.51 billion, savings \$2.51 billion, and business \$2.47 billion. Loan exposure sits at \$3.55 billion for consumer accounts and \$1.74 billion for corporate accounts.

Table 2: Total Balance and Loan amount by customer type

	Account type	Total Balance (USD)	Total Loan amount (USD)
Consumer	Checking	2,510,437,746	3,547,287,757
	Savings	2,509,275,452	
Corporate	Business	2,474,425,544	1,738,856,033

The clearest concentration risk number is this: the **top 10 accounts by balance hold a combined \$1,999,909, just 0.267% of total portfolio balance**. In a real bank you'd expect a handful of large corporate or high-net-worth accounts to carry a disproportionate share of total balance, here, no single account or small cluster of accounts comes close to dominating exposure, so concentration risk at the account level is effectively negligible.

Table 3: 10 merchants with greatest inflows from our customers

Merchant name	outflows to merchant
Smith Ltd	9,934,738.56
Johnson Group	9,086,620.40
Smith Inc	8,884,417.07
Smith and Sons	6,224,328.87
Johnson LLC	6,158,857.98
Miller PLC	6,112,326.12
Martin Ltd	5,935,659.34
Davis PLC	5,395,530.01
Jackson Ltd	5,243,459.48
Johnson Ltd	5,212,420.79

Merchant exposure tells the same story. The **top 10 merchants by out customers' outflows**, which are led by Smith Ltd at \$9.93 million, top out in the single-digit millions each, **nowhere near large enough** to represent meaningful counterparty concentration against a \$7.5 billion balance base. The **HHI Index for the merchant base comes out to 2.01**, confirming an **unconcentrated market**: no merchant or small group of merchants accounts for an outsized share of transaction flow.

Taken together, this reinforces the pattern from Sections 2 and 3: the data was generated to spread risk uniformly across accounts, customers, and merchants, with no organic clustering anywhere. That's useful for practicing concentration-risk methodology, but it also means concentration risk, as measured here, tells you nothing about how the dataset would behave in a real, imperfectly distributed portfolio.

Section 4: Credit Risk Analysis (your strongest section)

Here's the revised paragraph with the bands pulled out of the sentence:

This section asks the real question: does interest rate pricing in this dataset actually reflect credit risk? The answer is no. Weighted average interest rate by credit score band:

- Above 800: 8.56%
- 701-800: 8.46%
- 601-700: 8.65%
- 501-600: 8.50%
- Below 501: 8.51%

That's a spread of just **19 basis points across the entire credit spectrum**, with no consistent direction, a borrower with a score lesser than 501 pays essentially the same rate as a borrower above 800. In real lending records, you'd expect a meaningful risk premium, often several hundred basis points, between the riskiest and safest tiers. That gap is absent here, and it holds at the segment level too: **business and consumer loans stand at an identical 8.53% weighted average**. Rates in this dataset appear to be assigned independently of credit score, which is consistent with the uniform, synthetic generation pattern already established previously, but it unfortunately means this dataset can't be used to validate or build a real risk-based pricing model.

To size exposure to weak credits directly, I defined **high-risk loans** as those with a **credit score below 600 and an interest rate above 13.5%** (the top end of the observed rate range, given the dataset's max rate is 15%). By that definition, **6.5% are flagged high-risk** (1,950 of 30,000 loans), representing **\$287.2 million** of the \$4.51 billion in total loan exposure, or **6.4% of the portfolio** by dollar value. The fact that risk-weighted exposure (6.4%) tracks loan count (6.5%) almost exactly is itself

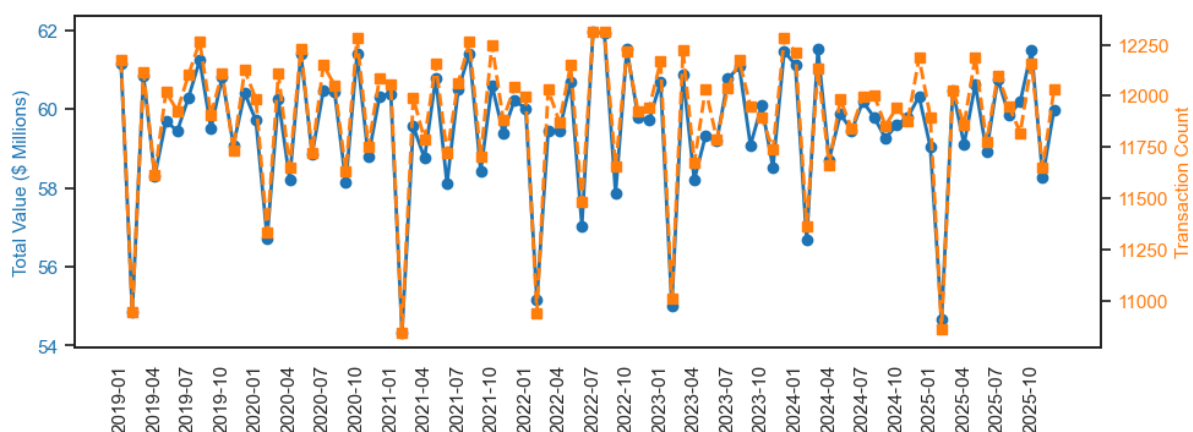
notable, in a real portfolio you'd expect these two percentages differ significantly, since risky loans aren't evenly sized. I also **flagged customers by loan-to-balance ratio**, using 0.85 as the outlier threshold: **21,257 of 50,000 customers (42.5%)** cross it, an unusually high share that again points to loan amounts being generated independently of account balances rather than sized to a customer's actual financial capacity.

Section 5: Transaction Behavior

Monthly transaction activity is broadly stable across the dataset, with one recurring exception: every January shows a dip in both transaction count and total amount compared to the rest of the year. Outside that pattern, volume is fairly uniform, the standard deviation across months is 328 transactions in count and \$1,653,748.80 in total amount, both small relative to the overall monthly averages, consistent with the evenly-generated activity noted throughout this report.

Two simple heuristics were run to flag unusual account behavior, note that these are basic pattern checks, not a fraud detection model. First, dormant accounts: looking at accounts with a positive balance, 29,307 have had no transactions in the last six months (measured back from December 31, 2025, the end of the data), and 434 have never had a single transaction. Second, same-day activity spikes: no account had five or more transactions on the same day at any point in the dataset. That absence is itself a little unrealistic, even ordinary behavior like a shopping trip or a weekend away would typically produce a same-day cluster of five or more transactions for some customers, so this is another marker of the data's synthetic origin rather than a clean bill of health.

Figure 6: Monthly Transactions: Total value transferred and Count of transactions



Section 6: Key Findings

This section distills the report's findings into the five takeaways that matter most, each grounded in a specific number from the analysis conducted priorly.

1. The **dataset is synthetic**, not real bank data, transaction amounts follow a **perfectly uniform distribution** instead of Benford's Law, account-level concentration metrics (Gini coefficient of 0.333) are identical across account types, and merchant concentration (HHI of 2.01) shows no natural clustering, patterns that don't occur in organic financial data.
2. **Interest rate pricing does not track credit risk** in this dataset, the weighted average rate spans just 19 basis points across all five credit score bands (8.46%-8.65%), far too narrow to reflect meaningful risk-based pricing.
3. **Portfolio exposure is highly diversified**, with no concentration risk at the account or merchant level, the top 10 accounts by balance hold only 0.267% of total portfolio balance, and the top merchant HHI of 2.01 confirms an unconcentrated counterparty base.
4. Despite flat pricing, **\$287.2 million sits in loans flagged high-risk** (6.4% of total loan exposure) under a credit score below 600 and interest rate above 13.5% threshold, and 42.5% of customers (21,257) exceed a 0.85 loan-to-balance ratio, both signal a portfolio that looks risky by volume even though pricing doesn't reflect it.
5. **Possible data quality issues predate any analysis**, city and country are null in every row, and 12.6% of customers share a full name with another customer, many accounts also share the same email adress, all of which limit what conclusions can be drawn about geography or true customer counts.

Taken together, these findings point to the same underlying conclusion from two directions: the data's statistical fingerprints (uniform distributions, flat pricing, near-identical concentration metrics) confirm it was synthetically generated, while the loan-level numbers (high-risk exposure, elevated loan-to-balance ratios) show that even synthetic data can still surface the kind of risk signals a real credit risk process would need to catch. The next section covers what this means for how far these results should be trusted, and what a real dataset would let this analysis do differently.

Section 7: Limitations & What I'd Do With More Data

This analysis has a clear boundary, and it's worth stating exactly where it sits. The dataset has **no default flag, no repayment or delinquency history, and no field indicating whether a loan is active or closed**, which is why active status was assumed for every loan in Section 1.

Without that, **none of the standard outputs can be built here** (credit risk outputs, probability of default, expected loss, delinquency rates), what this report offers instead is exposure sizing and pricing-consistency checks, which is a meaningfully different (and narrower) thing.

The dataset also has **no branch-to-customer or branch-to-transaction linkage**, so branch-level risk or performance can't be assessed despite branch data being present, and geographic analysis is similarly limited by null city and country fields at the customer level.

With a real or more complete dataset, the next steps would be building an actual expected-loss model using default flags and loss-given-default data, running a proper delinquency and vintage analysis by loan origination cohort, and linking transactions to branches to see whether risk or activity concentrates geographically.

Section 8: Data Sources & AI notice

Data sources

The dataset used for this report is ***Synthetic Banking Dataset (CSV, SQL, SQLite)*** by author *Akram Belhadi*. It was published in February earlier this year at the URL below:

<https://www.kaggle.com/datasets/akrambelha/synthetic-banking-dataset-csv-sql-sqlite>

AI notice

Portions of this report's written text were edited with the assistance of AI (Claude, Anthropic), based on my own text, notes, SQL queries, analysis, and findings. All data collection, calculations, and interpretation of results are my own work. AI assistance was limited to **improving clarity and phrasing** of the written sections.

Section 9: Appendix - SQL Queries & Python

The queries below are a representative sample of the SQL used to produce the figures and findings throughout this report, selected to show a range of techniques: window functions, multi-step CTEs, weighted aggregations, and threshold-based filtering. Each query is labeled with the section it supports. The full query set used for this analysis is longer than what's shown here; these were chosen as the clearest examples of the underlying method.

For more info on the structure of the relational database, check *Figure 1* on the second page of this document.

1. SQL: Loan to balance ratio per customer

```
-- loan to balance ratio per customer

WITH account_data AS (          -- agg accounts
    SELECT
        SUM(balance_usd) AS total_balance_usd,
        customer_id
    FROM accounts
    GROUP BY customer_id
), loan_data AS (              -- agg loans
    SELECT
        SUM(loan_amount) AS total_loan_amount,
        customer_id
    FROM loans
    GROUP BY customer_id
), customer_data AS(
    SELECT                  -- join to customers
        c.customer_id,
        COALESCE(a.total_balance_usd, 0) AS total_balance_usd,
        COALESCE(l.total_loan_amount, 0) AS total_loan_amount
    FROM customers c
    LEFT JOIN account_data a ON a.customer_id=c.customer_id
    LEFT JOIN loan_data l ON l.customer_id=c.customer_id
), ratio_data AS (
    SELECT
        customer_id,
        CASE
            WHEN total_balance_usd IS NOT 0
            THEN ROUND((total_loan_amount / total_balance_usd),2)
            ELSE 99.99
```

```
        END AS loan_to_balance_ratio
    FROM customer_data
    ORDER BY loan_to_balance_ratio
)
SELECT '-', loan_to_balance_ratio FROM ratio_data
WHERE loan_to_balance_ratio < 0.85;
```

2. SQL: dormant accounts (balance > 0, no transactions in last 6 months)

```
WITH transaction_data AS (
    SELECT MAX(transaction_date) AS most_recent_transaction_date,
           account_id,
           CASE
               WHEN MAX(transaction_date) >= '2025-07-01'
                    AND MAX(transaction_date) <= '2025-12-31' THEN 1
               ELSE 0
           END AS has_recent_transactions,
           CASE
               WHEN MAX(transaction_date) <= '2025-12-31' THEN 1
               ELSE 0
           END AS has_transactions
    FROM transactions
    GROUP BY account_id
), account_data AS (
    SELECT has_recent_transactions, has_transactions FROM
transaction_data t
    LEFT JOIN accounts a ON a.account_id=t.account_id
    WHERE balance_usd>0
)
SELECT COUNT(1) FROM account_data
WHERE has_transactions IS NOT 1;
```

3. SQL: Same-day spike detection: account-days with 5+ transactions

```
SELECT
    account_id,
    DATE(transaction_date) AS transaction_day,
```

```
COUNT(transaction_id) AS transaction_count,  
SUM(amount_usd) AS total_amount_usd  
FROM transactions  
GROUP BY  
    account_id,  
    DATE(transaction_date)  
HAVING  
    COUNT(transaction_id) >= 5  
ORDER BY  
    transaction_count DESC,  
    transaction_day DESC;
```

4. SQL: loan amount, number of loans, weighted average interest rate by credit score tiers

```
WITH Credit_Tiers AS (  
    SELECT  
        l.loan_id,  
        l.loan_amount,  
        l.interest_rate,  
        CASE  
            WHEN c.credit_score >= 750 THEN '1. Prime (750+)'  
            WHEN c.credit_score BETWEEN 700 AND 749 THEN '2. Near  
Prime (700-749)'  
            WHEN c.credit_score BETWEEN 650 AND 699 THEN '3. Subprime  
(650-699)'  
            ELSE '4. High Risk (<650)'  
        END AS credit_segment  
    FROM loans l  
    JOIN customers c ON l.customer_id = c.customer_id  
)  
SELECT  
    credit_segment,  
    COUNT(loan_id) AS total_loans,  
    SUM(loan_amount) AS total_loan_amount,  
    ROUND(SUM(interest_rate * loan_amount) / SUM(loan_amount), 4) AS
```

```
weighted_avg_interest_rate
FROM Credit_Tiers
GROUP BY credit_segment
ORDER BY credit_segment;
```

5. Python: Creation of *Figure 6: Monthly Transactions: Total value transferred and Count of transactions*, on page 8

```
# Load dataset in dataframe
df =
pd.read_csv("/Users/tinelisec/Documents/Career/Learning/Portfolio
projects/01_Portfolio & Credit Risk Analysis of a Synthetic Retail
Lending Dataset/data/monthly_transaction_activity.csv")

# Initialize figure and primary Y-axis (Total Value)
fig, ax1 = plt.subplots(figsize=(10, 5), facecolor='white')
ax1.set_facecolor('white')

color = '#1f77b4'
ax1.set_xlabel('Month of Transaction', fontsize=11, labelpad=15)
ax1.set_ylabel('Total Value ($ Millions)', color=color, fontsize=11)
line1 = ax1.plot(
    df['month_of_transaction'],
    df['total'] / 1e6,
    color=color,
    marker='o',
    linewidth=2,
    label='Total ($ Millions)'
)
ax1.tick_params(axis='y', labelcolor=color)

# Secondary Y-axis (Transaction Count)
ax2 = ax1.twinx()
color = '#ff7f0e'
ax2.set_ylabel('Transaction Count', color=color, fontsize=11)
```

```
line2 = ax2.plot(
    df['month_of_transaction'],
    df['count'],
    color=color,
    marker='s',
    linewidth=2,
    linestyle='--',
    label='Transaction Count'
)
ax2.tick_params(axis='y', labelcolor=color)

# Filter x-ticks to display 4 quarterly months per year ('01', '04',
'07', '10')
q_months = ['01', '04', '07', '10']
selected_ticks = [i for i, m in enumerate(df['month_of_transaction'])
if m.split('-')[1] in q_months]
selected_labels = [df['month_of_transaction'].iloc[i] for i in
selected_ticks]

# Set ticks and rotate labels vertically (90 degrees)
ax1.set_xticks(selected_ticks)
ax1.set_xticklabels(selected_labels, rotation=90, ha='center')

# Combined legend and layout
lines = line1 + line2
labels = [l.get_label() for l in lines]
ax1.legend(lines, labels, loc='upper center', bbox_to_anchor=(0.5,
-0.32), ncol=2, frameon=False)

plt.title('Monthly Transactions: Total Value vs Count (2019)',
fontsize=14, fontweight='bold', pad=15)
plt.tight_layout()
plt.show()
```

6. Python: Creation of *Figure 5: Account tenure, too uniform to be realistic*, on page 5

```
# 1. Load the data from your CSV file
df = pd.read_csv('./data/account_creation.csv')

# 2. Aggregate if your CSV contains individual rows
if 'customer_count' not in df.columns and 'month_of_creation' in
df.columns:
    df =
df.groupby('month_of_creation').size().reset_index(name='customer_cou
nt')

df = df.sort_values('month_of_creation').reset_index(drop=True)

# 3. Generate the bar chart
fig, ax = plt.subplots(figsize=(10, 6))
ax.bar(df['month_of_creation'], df['customer_count'],
color='#3498db', edgecolor='black')

# 4. Filter X-axis to only show every 4th month label
ax.xaxis.set_major_locator(ticker.IndexLocator(base=4, offset=0))

# 5. Visual formatting
plt.title('Customer Account Registrations by Quarter / Interval',
fontsize=14, fontweight='bold')
plt.xlabel('Month of Creation (YYYY-MM)', fontsize=12)
plt.ylabel('Number of New Accounts', fontsize=12)
plt.xticks(rotation=45)
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()

# 6. Save and view the plot
plt.savefig('customer_registrations_chart.png', dpi=300)
plt.show()
```